

Ex.No : 1 IMPLEMENTATION OF DECISION TREES FOR REAL WORLD : WORLD PROBLEM

Aim:

To Implement the concept of decision trees with suitable data set from real world problem and classify the data set to produce new sample.

Algorithm:

Step 1: Start the program

Step 2: Import the module which are needed

Step 3: Load the dataset in the .csv format.

Step 4: Perform various basic operations with loaded dataset to avoid missing values

Step 5: Implement the decision tree algorithm to solve real world problem

Step 6: Print the obtained results

Step 7: Stop the program

Program

```
import pandas
from sklearn import tree
from sklearn.tree import DecisionTreeClassifier
import matplotlib.pyplot as plt
df = pandas.read_csv("C:\\Users\\IT\\Downloads\\data - Sheet1.csv")
d = {'UK': 0, 'USA': 1, 'N': 2}
df['Nationality'] = df['Nationality'].map(d)
d = {'YES': 1, 'NO': 0}
df['Go'] = df['Go'].map(d)
features = ['Age', 'Experience', 'Rank', 'Nationality']
X = df[features]
y = df['Go']
```

```
dtree = DecisionTreeClassifier()  
dtree = dtree.fit(X, y)  
tree.plot_tree(dtree, feature_names=features)  
plt.show()
```

Output:

Result:

Ex.No : 2 DETECTION OF SPAM MAILS USING SUPPORT
Date : VECTOR MACHINE

Aim:

To implement the detection of spam mails using Support Vector Machine

Algorithm

Step 1: Start the program

Step 2: Import the module which are needed

Step 3: Load the dataset in the .csv format

Step 4: Perform various basic operations with loaded dataset to avoid missing values

Step 5: Implement the support vector machine algorithm to detect the spam mails

Step 6: Print the obtained results

Step 7: Stop the program

Program

```
import numpy as np

import pandas as pd

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.model_selection import GridSearchCV

from sklearn import svm

data = pd.read_csv("C:\\Users\\IT\\Downloads\\spam.csv")

print(data)

print(data.info())

X=data['EmailText'].values

print(X)
```

```
Y=data['Label'].values
```

```
print(Y)
```

```
from sklearn.svm import SVC
```

```
classifier = SVC(kernel = 'rbf', random_state = 10)
```

```
classifier.fit(X_train, y_train)
```

```
print(classifier.score(X_test,y_test))
```

Output

Result:

EX. NO : 3 IMPLEMENTATION OF FACIAL RECOGNITION
Date : APPLICATION WITH ARTIFICIAL NEURAL NETWORK

Aim:

To implement facial recognition application with Artificial Neural Network.

Algorithm

Step 1: Start the program

Step 2: Import the module which are needed

Step 3: Load the dataset with images

Step 4: Perform various basic operations with loaded dataset to avoid missing values

Step 5: Implement the artificial neural network to recognize the faces

Step 6: Print the obtained results

Step 7: Stop the program

Program

```
import numpy as np
import tensorflow as tf
import keras
from keras.models import Sequential
from keras.layers import Conv2D,MaxPooling2D,Dense,Flatten,Dropout
import matplotlib.pyplot as plt
from keras.layers.normalization import BatchNormalization
from keras_preprocessing import image
from tensorflow.keras.preprocessing.image import ImageDataGenerator
train_dir="../input/face-recognition-dataset/Original Images/Original Images/"
generator = ImageDataGenerator()
train_ds = generator.flow_from_directory(train_dir,target_size=(224, 224),batch_size=3
2)
classes = list(train_ds.class_indices.keys())
```

```
model = Sequential()

model.add(Conv2D(32, kernel_size = (3, 3), activation='relu', input_shape=(224,224,3))
)

model.add(MaxPooling2D(pool_size=(2,2)))

model.add(BatchNormalization())

model.add(Conv2D(64, kernel_size=(3,3), activation='relu'))

model.add(MaxPooling2D(pool_size=(2,2)))

model.add(BatchNormalization())

model.add(Conv2D(64, kernel_size=(3,3), activation='relu'))

model.add(MaxPooling2D(pool_size=(2,2)))

model.add(BatchNormalization())

model.add(Conv2D(96, kernel_size=(3,3), activation='relu'))

model.add(MaxPooling2D(pool_size=(2,2)))

model.add(BatchNormalization())

model.add(Conv2D(32, kernel_size=(3,3), activation='relu'))

model.add(MaxPooling2D(pool_size=(2,2)))

model.add(BatchNormalization())

model.add(Dropout(0.2))

model.add(Flatten())

model.add(Dense(128, activation='relu'))

#model.add(Dropout(0.3))

model.add(Dense(len(classes),activation='softmax'))

model.compile(loss = 'categorical_crossentropy',optimizer = 'adam', metrics = ["accuracy"])

model.summary()

history = model.fit(train_ds,epochs= 30, batch_size=32)

plt.plot(history.history['accuracy'])
```

```
plt.plot(history.history['loss'])

plt.xlabel('Time')

plt.legend(['accuracy', 'loss'])

plt.show()

def predict_image(image_path):

    img = image.load_img(image_path, target_size=(224,224,3))

    plt.imshow(img)

    plt.show()

    x = image.img_to_array(img)

    x = np.expand_dims(x, axis=0)

    images = np.vstack([x])

    pred = model.predict(images, batch_size=32)

    print("Actual: "+(image_path.split("/")[-1]).split("_")[0])

    print("Predicted: "+classes[np.argmax(pred)])

    predict_image("../input/face-recognition-dataset/Original Images/Original Images/Brad Pitt/Brad Pitt_102.jpg")
```

Output:

Result:

Ex.No:4

IMPLEMENTATION OF AMAZON TOOLKIT:

Date :

SAGEMAKER

Aim:

To implement the Amazon Toolkit Sagemaker

Procedure:

1. Setting up Sagemaker

2. Uploading Data to S3 Storage

3. Writing the Machine Learning Pipeline using Python

4. Creating notebook interface for Amazon Sagemaker

5. Creating Role setup for Amazon Sagemaker

6. Notebook created in Amazon Sagemaker

7. Selecting S3 storage in AWS

8. Creating Custom Dataset in S3 storage in AWS

9. Data uploaded in the S3 storage in AWS

10. Creating Notebook in Amazon Sagemaker

11. Python code for importing the data set from S3 storage and using in the machine learning pipeline in Amazon Sagemaker notebook

12. Amazon's proprietary machine learning models available in Amazon Sagemaker

Result:

Ex.No:5 IMPLEMENTATION OF CHARACTER RECOGNITION
Date : USING MULTILAYER PERCEPTRON

Aim:

To implement the character recognition using Multilayer Perceptron

Algorithm

Step 1: Start the program

Step 2: Import the module which are needed

Step 3: Load the dataset with images

Step 4: Perform various basic operations with loaded dataset to avoid missing values

Step 5: Implement the multilayer perceptron to recognize the character

Step 6: Print the obtained results

Step 7: Stop the program

Program

```
# importing modules
import tensorflow as tf
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Activation
import matplotlib.pyplot as plt
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
# Cast the records into float values
x_train = x_train.astype('float32')
x_test = x_test.astype('float32')
```

```

# normalize image pixel values by dividing
# by 255
gray_scale = 255
x_train /= gray_scale
x_test /= gray_scale
print("Feature matrix:", x_train.shape)
print("Target matrix:", x_test.shape)
print("Feature matrix:", y_train.shape)
print("Target matrix:", y_test.shape)
fig, ax = plt.subplots(10, 10)
k = 0
for i in range(10):
    for j in range(10):
        ax[i][j].imshow(x_train[k].reshape(28, 28), aspect='auto')
        k += 1
plt.show()
model = Sequential([
    # reshape 28 row * 28 column data to 28*28 rows
    Flatten(input_shape=(28, 28)),
    # dense layer 1
    Dense(256, activation='sigmoid'),
    # dense layer 2
    Dense(128, activation='sigmoid'),
    # output layer
    Dense(10, activation='sigmoid'),])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train, epochs=10, batch_size=2000, validation_split=0.2)
results = model.evaluate(x_test, y_test, verbose = 0)
print('test loss, test acc:', results)

```

Output:

Result:

**Ex.No : 6 IMPLEMENTATION OF THE NON-PARAMETRIC LOCALLY
Date : WEIGHTED REGRESSION ALGORITHM**

Aim:

To implement the non-parametric locally weighted regression algorithm

Algorithm

Step 1: Start the program

Step 2: Import the module which are needed

Step 3: Load the dataset

Step 4: Perform various basic operations with loaded dataset to avoid missing values

Step 5: Implement the locally weighted regression algorithm

Step 6: Print the obtained results

Step 7: Stop the program

Program

```
import numpy as np
```

```
from bokeh.plotting import figure, show, output_notebook
```

```
from bokeh.layouts import gridplot
```

```
from bokeh.io import push_notebook
```

```
def local_regression(x0, X, Y, tau):# add bias term
```

```
x0 = np.r_[1, x0] # Add one to avoid the loss in information
```

```
X = np.c_[np.ones(len(X)), X]
```

```
# fit model: normal equations with kernel
```

```
xw = X.T * radial_kernel(x0, X, tau) # XTranspose * W
```

```
beta = np.linalg.pinv(xw @ X) @ xw @ Y #@ Matrix Multiplication or Dot Product
```

predict value

```

return x0 @ beta # @ Matrix Multiplication or Dot Product for prediction

def radial_kernel(x0, X, tau):

    return np.exp(np.sum((X - x0) ** 2, axis=1) / (-2 * tau * tau))

# Weight or Radial Kernel Bias Function

n = 1000

# generate dataset

X = np.linspace(-3, 3, num=n)

print("The Data Set ( 10 Samples) X :\n",X[1:10])

Y = np.log(np.abs(X ** 2 - 1) + .5)

print("The Fitting Curve Data Set (10 Samples) Y :\n",Y[1:10])

# jitter X

X += np.random.normal(scale=.1, size=n)

print("Normalised (10 Samples) X :\n",X[1:10])

domain = np.linspace(-3, 3, num=300)

print(" Xo Domain Space(10 Samples) :\n",domain[1:10])

def plot_lwr(tau):

    # prediction through regression

    prediction = [local_regression(x0, X, Y, tau) for x0 in domain]

    plot = figure(plot_width=400, plot_height=400)

    plot.title.text='tau=%g' % tau

    plot.scatter(X, Y, alpha=.3)

    plot.line(domain, prediction, line_width=2, color='red')

    return plot

show(gridplot([ [plot_lwr(10.), plot_lwr(1.)],[plot_lwr(0.1), plot_lwr(0.01)]]))

```

Output

Result:

Ex.No:7 IMPLEMENTATION OF SENTIMENT ANALYSIS USING
Date : RANDOM FOREST OPTIMIZATION ALGORITHM

Aim:

To implement the sentiment analysis using Random Forest optimization Algorithm

Algorithm:

Step 1: Start the program

Step 2: Import the module which are needed

Step 3: Load the dataset with csv format

Step 4: Perform various basic operations with loaded dataset to avoid missing values

Step 5: Implement the Random Forest algorithm to analysis the sentiments

Step 6: Print the obtained results

Step 7: Stop the program

Program:

```
import numpy as np # linear algebra
```

```
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
import re
```

```
import nltk
```

```
# Input data files are available in the read-only &quot;../input/&quot; directory
```

```
# For example, running this (by clicking run or pressing Shift+Enter) will list all files  
under the input directory
```

```

import os

for dirname, _, filenames in os.walk('kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

imdb=pd.read_csv('C:\\Users\\produ\\Downloads\\new\\Train.csv')

print(imdb.info())

print(imdb.shape)

print(imdb.head(10))

imdb['label'].value_counts().plot.pie(figsize=(6,6),title='Distribution
of reviews per

sentiment',labels=[' ',' ',' ',' '],autopct='%1.1f%%')

labels=['Positive','Negative']

plt.legend(labels,loc=3)

plt.gca().set_aspect('equal')

from sklearn.model_selection import train_test_split

features = imdb.drop('label',axis=1)

labels = imdb['label']

X_train, X_test, y_train, y_test = train_test_split(features,labels,test_size = 0.90,
random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X_train,y_train,test_size = 0.5,
random_state=42)

X_val, X_test, y_val, y_test = train_test_split(X_test,y_test,test_size = 0.5,
random_state=42)

print('Data distribution:\n- Train: { } \n- Validation: { } \n-
Test:{ }'.format(len(y_train),len(y_val),len(y_test)))

```

Output:

Result:

Ex.No: 8 CONSTRUCTION OF A BAYESIAN NETWORK CONSIDERING

Date : MEDICAL DATA

Aim:

To write a python program to construct a Bayesian network considering Medical data

Algorithm:

Step 1: Start the program

Step 2: Import the module which are needed

Step 3: Load the dataset with csv format

Step 4: Perform various basic operations with loaded dataset to avoid missing values

Step 5: Implement the Bayesian Network considering medical data

Step 6: Print the obtained results

Step 7: Stop the program

Program:

```
import pandas as pd
from pgmpy.estimators import MaximumLikelihoodEstimator
from pgmpy.models import BayesianNetwork
from pgmpy.inference import VariableElimination
data = pd.read_csv("C:\\Users\\produ\\Desktop\\ds4.csv")
heart_disease = pd.DataFrame(data)
print(heart_disease)
model = BayesianNetwork([
    ('age', 'Lifestyle'),
    ('Gender', 'Lifestyle'),
    ('Family', 'heartdisease'),
    ('diet', 'cholesterol'),
    ('Lifestyle', 'diet'),
```

```

('cholesterol', 'heartdisease'),
('diet', 'cholesterol']])
model.fit(heart_disease, estimator=MaximumLikelihoodEstimator)
HeartDisease_infer = VariableElimination(model)
print('For Age enter SuperSeniorCitizen:0, SeniorCitizen:1, MiddleAged:2, Youth:3,
Teen:4')
print('For Gender enter Male:0, Female:1')
print('For Family History enter Yes:1, No:0')
print('For Diet enter High:0, Medium:1')
print('for LifeStyle enter Athlete:0, Active:1, Moderate:2, Sedentary:3')
print('for Cholesterol enter High:0, BorderLine:1, Normal:2')
q = HeartDisease_infer.query(variables=['heartdisease'], evidence={
    'age': int(input('Enter Age: ')),
    'Gender': int(input('Enter Gender: ')),
    'Family': int(input('Enter Family History: ')),
    'diet': int(input('Enter Diet: ')),
    'Lifestyle': int(input('Enter Lifestyle: ')),
    'cholesterol': int(input('Enter Cholesterol: '))})
print(q)

```

Output:

Result:

Ex.No: 9 IMPLEMENTATION OF ONLINE FRAUD DETECTION
Date : USING BEST MACHINE LEARNING ALGORITHM

Aim:

To implement online fraud detection using best machine learning algorithm

Algorithm:

Step 1: Start the program

Step 2: Import the module which are needed

Step 3: Load the dataset with csv format

Step 4: Perform various basic operations with loaded dataset to avoid missing values

Step 5: Implement the Random Forest algorithm for online fraud detection

Step 6: Print the obtained results

Step 7: Stop the program

Program:

```
import pandas as pd
data=pd.read_csv("credit.csv",encoding='windows-1252').dropna()
print(data)
X=data.drop(['Class'],axis=1)
print(X)
Y=data['Class']
print(Y)
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.30)
print(x_train)
print(x_test)
print(y_train)
print(y_test)
from sklearn.ensemble import RandomForestClassifier
```

```
clf=RandomForestClassifier(n_estimators=100)
clf.fit(x_train,y_train)
y_pred=clf.predict(x_test)
print(y_pred)
from sklearn.metrics import accuracy_score
acc1=accuracy_score(y_pred,y_test)
print(acc1)
```

Output:

Result:

Ex.No:10

MINI PROJECT

Date :

Aim:

To write a python program to implement a socially relevant problem which needs machine learning solution

Algorithm:

Step 1: Start the program

Step 2: Import the module which are needed

Step 3: Load the dataset with csv format

Step 4: Perform various basic operations with loaded dataset to avoid missing values

Step 5: Implement the Random Forest algorithm for online fraud detection

Step 6: Print the obtained results

Step 7: Stop the program

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
from matplotlib.pylab import rcParams
rcParams['figure.figsize']=20,10
from keras.models import Sequential
from keras.layers import LSTM,Dropout,Dense
from sklearn.preprocessing import MinMaxScaler
df=pd.read_csv("NSE-TATA.csv")
```

```

df.head()

df["Date"]=pd.to_datetime(df.Date,format="%Y-%m-%d")
df.index=df['Date']

plt.figure(figsize=(16,8))
plt.plot(df["Close"],label='Close Price history')

data=df.sort_index(ascending=True,axis=0)
new_dataset=pd.DataFrame(index=range(0,len(df)),columns=['Date','Close'])
for i in range(0,len(data)):
    new_dataset["Date"][i]=data["Date"][i]
    new_dataset["Close"][i]=data['Close'][i]
    scaler=MinMaxScaler(feature_range=(0,1))
    final_dataset=new_dataset.values
    train_data=final_dataset[0:987,:]
    valid_data=final_dataset[987:,:]
    new_dataset.index=new_dataset.Date
    new_dataset.drop("Date",axis=1,inplace=True)
    scaler=MinMaxScaler(feature_range=(0,1))
    scaled_data=scaler.fit_transform(final_dataset)
    x_train_data,y_train_data=[],[]
    for i in range(60,len(train_data)):
        x_train_data.append(scaled_data[i-60:i,0])
        y_train_data.append(scaled_data[i,0])
    x_train_data,y_train_data=np.array(x_train_data),np.array(y_train_data)
    x_train_data=np.reshape(x_train_data,(x_train_data.shape[0],x_train_data.shape[1],1))

    lstm_model=Sequential()

    lstm_model.add(LSTM(units=50,return_sequences=True,input_shape=(x_train_data.
    shape[1],1)))

    lstm_model.add(LSTM(units=50))
    lstm_model.add(Dense(1))

```

```

inputs_data=new_dataset[len(new_dataset)-len(valid_data)-60:].values
inputs_data=inputs_data.reshape(-1,1)
inputs_data=scaler.transform(inputs_data)
lstm_model.compile(loss='mean_squared_error',optimizer='adam')
lstm_model.fit(x_train_data,y_train_data,epochs=1,batch_size=1,verbose=2)
X_test=[]
for i in range(60,inputs_data.shape[0]):
    X_test.append(inputs_data[i-60:i,0])
X_test=np.array(X_test)
X_test=np.reshape(X_test,(X_test.shape[0],X_test.shape[1],1))
predicted_closing_price=lstm_model.predict(X_test)
predicted_closing_price=scaler.inverse_transform(predicted_closing)
lstm_model.save("saved_model.h5")

```

Output:

Result:

Ex.No:11 IMPLEMENTATION OF SENTIMENTAL ANALYSIS USING
Date : LEXICON CLASSIFICATION ALGORITHM

Aim:

To write a python program to implement a sentimental analysis using Lexicon Classification Algorithm

Algorithm:

Step 1: Start the program

Step 2: Import the module which are needed

Step 3: Load the dataset with csv format

Step 4: Perform various basic operations with loaded dataset to avoid missing values

Step 5: Implement the Lexicon Classification Algorithm for sentimental analysis

Step 6: Print the obtained results

Step 7: Stop the program

Program

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

import re

%matplotlib inline

import warnings

warnings.filterwarnings("ignore")

import os

print(os.listdir("../input"))
```

```
pd.set_option('display.max_columns',None)

US_comments=pd.read_csv('../input/youtube/UScomments.csv',error_bad_lines=False)

US_videos=pd.read_csv('../input/youtube/USvideos.csv',error_bad_lines=False)

US_videos.head()

US_videos.shape

US_videos.nunique()

US_videos.info()

US_videos.head()

US_comments.head()

US_comments.shape

US_comments.isnull().sum()

US_comments.dropna(inplace=True)

US_comments.isnull().sum()

US_comments.shape

US_comments.nunique()

US_comments.info()

US_comments.drop(41587,inplace=True)

US_comments=US_comments.reset_index().drop('index',axis=1)

US_comments.likes=US_comments.likes.astype(int)

US_comments.replies=US_comments.replies.astype(int)

US_comments.head()

US_comments['comment_text']=US_comments['comment_text'].str.replace("[^a-zA-Z#]",", " " ")
```

```
US_comments['comment_text']=US_comments['comment_text'].apply(lambda x: '
.join([w for w in x.split() if len(w)>3]))

US_comments['comment_text']=US_comments['comment_text'].apply(lambda
x:x.lower())

tokenized_tweet=US_comments['comment_text'].apply(lambda x:x.split())

tokenized_tweet.head()

from nltk.stem import WordNetLemmatizer

from nltk.corpus import stopwords

wnl = WordNetLemmatizer()

tokenized_tweet.apply(lambda x: [wnl.lemmatize(i) for i in x if i not in
set(stopwords.words('english'))])

tokenized_tweet.head()

US_comments['comment_text'] = tokenized_tweet

import nltk

nltk.download('vader_lexicon')

from nltk.sentiment.vader import SentimentIntensityAnalyzer

sia = SentimentIntensityAnalyzer()

US_comments['Sentiment Scores'] = US_comments['comment_text'].apply(lambda
x:sia.polarity_scores(x)['compound'])

US_comments.head()

US_comments['Sentiment'] = US_comments['Sentiment Scores'].apply(lambda s :
'Positive' if s > 0 else ('Neutral' if s == 0 else 'Negative'))

US_comments.head()

US_comments.Sentiment.value_counts()

videos = []
```

```

for i in range(0,US_comments.video_id.nunique()):

    a = US_comments[(US_comments.video_id == US_comments.video_id.unique()[i])
    & (US_comments.Sentiment == 'Positive')].count()[0]

    b=US_comments[US_comments.video_id==US_comments.video_id.unique()[i]][['Sen
    timent']].value_counts().sum()

    Percentage = (a/b)*100

    videos.append(round(Percentage,2))

Positivity = pd.DataFrame(videos,US_comments.video_id.unique()).reset_index()

Positivity.columns = ['video_id','Positive Percentage']

Positivity.head()

channels = []

for i in range(0,Positivity.video_id.nunique()):

    channels.append(US_videos[US_videos.video_id ==
    Positivity.video_id.unique()[i]][['channel_title']].unique()[0])

    Positivity['Channel'] = channels

    Positivity.head()

    Positivity[Positivity['Positive Percentage'] == Positivity['Positive Percentage'].max()]

    Positivity[Positivity['Positive Percentage'] == Positivity['Positive Percentage'].min()]

    all_words = ' '.join([text for text in US_comments['comment_text']])

    from wordcloud import WordCloud

    wordcloud = WordCloud(width=800, height=500, random_state=21,
    max_font_size=110).generate(all_words)

    plt.figure(figsize=(10, 7))

    plt.imshow(wordcloud, interpolation="bilinear")

    plt.axis('off')

```

```
plt.show()

all_words_posi = ' '.join([text for text in
US_comments['comment_text'][(US_comments.Sentiment == 'Positive')]])

wordcloud_posi = WordCloud(width=800, height=500, random_state=21,
max_font_size=110).generate(all_words_posi)

plt.figure(figsize=(10, 7))

plt.imshow(wordcloud_posi, interpolation="bilinear")

plt.axis('off')

plt.show()

all_words_neg = ' '.join([text for text in
US_comments['comment_text'][(US_comments.Sentiment == 'Negative')]])

wordcloud_neg = WordCloud(width=800, height=500, random_state=21,
max_font_size=110).generate(all_words_neg)

plt.figure(figsize=(10, 7))

plt.imshow(wordcloud_neg, interpolation="bilinear")

plt.axis('off')

plt.show()

all_words_neu = ' '.join([text for text in
US_comments['comment_text'][(US_comments.Sentiment == 'Neutral')]])

wordcloud_neu = WordCloud(width=800, height=500, random_state=21,
max_font_size=110).generate(all_words_neu)

plt.figure(figsize=(10, 7))

plt.imshow(wordcloud_neu, interpolation="bilinear")

plt.axis('off')

plt.show()
```

Output

Result:

EX.NO : 12 IMPLEMENTATION OF AGGLOMERATIVE CLUSTERING

Date :

Aim:

To implement the agglomerative clustering

Algorithm:

- Step 1: Import the necessary libraries.
- Step 2: Import the dataset
- Step 3: Split the dataset for training and testing
- Step 4: Visualize the dendrogram
- Step 5: Plot the graph

Program:

```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
from sklearn.cluster import AgglomerativeClustering
import scipy.cluster.hierarchy as sch

dataset = pd.read_csv('Mall_Customers_data.csv')
X = dataset.iloc[:, [3, 4]].values

dendrogram = sch.dendrogram(sch.linkage(X, method='ward'))

model = AgglomerativeClustering(n_clusters=5, affinity='euclidean',
linkage='ward')
model.fit(X)

labels = model.labels_

plt.scatter(X[labels==0, 0], X[labels==0, 1], s=50, marker='o', color='red')
plt.scatter(X[labels==1, 0], X[labels==1, 1], s=50, marker='o', color='blue')

plt.scatter(X[labels==2, 0], X[labels==2, 1], s=50, marker='o', color='green')
plt.scatter(X[labels==3, 0], X[labels==3, 1], s=50, marker='o', color='purple')
plt.scatter(X[labels==4, 0], X[labels==4, 1], s=50, marker='o', color='orange')
plt.show()
```

Output

Result

Ex.NO:13 IMPLEMENTATION OF ENSEMBLE CLASSIFIER

Date:

Aim:

To implement the ensemble classifier algorithm.

Algorithm:

Step 1: Import the libraries.

Step 2:example of evaluating an adaboost ensemble for classification

Step 3:create the synthetic classification dataset

Step 4: configure the ensemble model

Step 5: configure the resampling method

Step 6: evaluate the ensemble on the dataset using the resampling

Step7: report ensemble performance

Program:

AdaBoosting Bagging

example of evaluating an adaboost ensemble for classification

from numpy import mean from numpy import std

from sklearn.datasets import make_classification from sklearn.model_selection import
cross_val_score

from sklearn.model_selection import RepeatedStratifiedKFold from sklearn.ensemble
import AdaBoostClassifier

create the synthetic classification dataset

X, y = make_classification(random_state=1)

configure the ensemble model

model = AdaBoostClassifier(n_estimators=50)

evaluate the ensemble on the dataset using the resampling method

n_scores = cross_val_score(model, X, y, scoring='accuracy',cv=cv, n_jobs=-1)

report ensemble performance

print('Mean Accuracy ADABOOST(Boosting): %.3f (%.3f)' %

(mean(n_scores),std(n_scores)))

Mean Accuracy ADABOOST(Boosting): 0.950 (0.072)

example of evaluating a bagging ensemble for classification

```
from numpy import mean from numpy import std
```

```
from sklearn.datasets import make_classification from sklearn.model_selection import  
cross_val_score from sklearn.model_selection import RepeatedStratifiedKFold from  
sklearn.ensemble import BaggingClassifier
```

create the synthetic classification dataset

```
X, y = make_classification(random_state=1)
```

configure the ensemble model

```
model = BaggingClassifier(n_estimators=50)
```

configure the resampling method

```
cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=1)
```

evaluate the ensemble on the dataset using the resampling method

```
n_scores = cross_val_score(model, X, y, scoring='accuracy', cv=cv, n_jobs=-1)
```

report ensemble performance

```
print('Mean Accuracy of BAGGING: %.3f (%.3f)' % (mean(n_scores), std(n_scores)))
```

Output:

Result: